

Documentación Técnica

Sistema Basado en Conocimiento - Diagnóstico DIAN / RNDC

Autores:

Deyvid Santiago Galarza González

Jean Paul Solano Hernández

Mayo 2026

Documentación Técnica

Sistema Basado en Conocimiento: Diagnóstico de Envíos DIAN / RNDC

Autores: Deyvid Santiago Galarza González · Jean Paul Solano Hernández

Asignatura: Inteligencia Artificial II - Proyecto Final

Institución: Fundación Universitaria Los Libertadores

Fecha: Mayo 2026

1. Introducción

1.1 Descripción del dominio

Las empresas de transporte y logística en Colombia están obligadas por ley a reportar sus operaciones a dos entidades gubernamentales:

- **DIAN** (Dirección de Impuestos y Aduanas Nacionales): recibe facturas electrónicas y notas crédito.
- **RNDC** (Registro Nacional de Despachos de Carga): recibe manifiestos de carga y remesas de transporte.

Cuando un documento es rechazado o el envío falla, el error puede tener múltiples causas: problemas de conectividad, certificados vencidos, datos mal diligenciados, servicios externos caídos, entre otros.

Identificar la causa correcta requiere conocimiento especializado que no siempre está disponible en el momento del fallo.

1.2 Justificación

Los errores en estos envíos generan:

- Multas por incumplimiento de obligaciones tributarias y de transporte.
- Bloqueos operativos que impiden la facturación.
- Retrasos en la cadena de pagos.

Automatizar el diagnóstico reduce el tiempo de resolución de horas a segundos y permite a usuarios no técnicos identificar y reportar el problema correctamente.

1.3 Alcance

El sistema cubre los errores más frecuentes en el envío de facturas, notas crédito, remesas y manifiestos:

Tipo de problema	Cubierto
Servicio DIAN o RNDC no disponible	[SI]

Problema de conectividad a internet	[SI]
Error en la integración API	[SI]
Certificado digital vencido	[SI]
Rango de numeración inválido	[SI]
Datos de entidades mal diligenciados	[SI]
Documento duplicado	[SI]
Casos de envío exitoso	[SI]
Bugs internos del software	[NO] (fuera de alcance)
Configuraciones de red empresarial complejas	[NO] (fuera de alcance)

2. Diseño del sistema

2.1 Arquitectura general

```

????????????????????????????????????????????
?      INTERFAZ WEB (app.py)      ?
?  Formulario -> Resultados -> ¿Por qué? ?
????????????????????????????????????????????
?      ? llama a
????????????????????????????????????????????
?  MOTOR DE INFERENCIA      ?
?  (motor_inferencia.py)    ?
?                             ?
?  encadenamiento_adelante() ?
?  porque()                 ?
?  explicar_razonamiento()   ?
?  validar_hechos() / aplicar_cwa() ?
????????????????????????????????????????????
?      ? consulta
????????????????????????????????????????????
?  BASE DE CONOCIMIENTO      ?
?  (base_conocimiento.py)    ?
?                             ?
?  PREDICADOS (28)           ?
?  REGLAS (35)               ?
?  DESCRIPCIONES_DIAGNOSTICO ?
????????????????????????????????????????????

```

2.2 Componentes principales

Componente	Archivo	Responsabilidad
Base de conocimiento	src/base_conocimiento.py	Define predicados, reglas, descripciones
Motor de inferencia	src/motor_inferencia.py	Aplica reglas y genera explicaciones
Interfaz web	src/app.py	Recoge síntomas, muestra resultados

2.3 Flujo de datos

```

    Usuario selecciona síntomas
        ?
app.py construye lista de hechos [(predicado, valor), ...]
        ?
motor_inferencia.validar_hechos() - verifica consistencia
        ?
motor_inferencia.aplicar_cwa() - completa hechos no provistos como False
        ?
motor_inferencia.encadenamiento_adelante() - aplica reglas iterativamente
        ?
Se derivan conclusiones con factores de certeza
        ?
app.py muestra resultados + botón ¿Por qué? por diagnóstico
        ?
motor_inferencia.porque() - explica cadena de razonamiento

```

3. Base de conocimiento

3.1 Predicados formalmente definidos

Todos los predicados son **unarios** con dominio {envíos de documentos electrónicos} y rango {True, False}.

Predicados de clasificación

Predicado	Significado	Tipo
Documento(x)	x es un documento electrónico	Síntoma
Factura(x)	x es una factura electrónica	Síntoma
NotaCredito(x)	x es una nota crédito	Síntoma
Remesa(x)	x es una remesa de transporte	Síntoma
Manifiesto(x)	x es un manifiesto de carga	Síntoma

Predicados de destino

Predicado	Significado
EnvioDian(x)	El documento x se envía a la DIAN
EnvioRNDC(x)	El documento x se envía al RNDC

Predicados de síntoma (observable)

Predicado	Significado
TieneInternet(x)	El equipo tiene conexión a internet al enviar x
ErrorConexion(x)	Se presentó error de conexión al enviar x
TimeoutEnvio(x)	El envío de x superó el tiempo de espera
FallaSOAP(x)	El servicio devolvió una falla SOAP al enviar x
DocumentoRechazado(x)	El servicio externo rechazó x
DocumentoDuplicado(x)	x ya había sido enviado antes
CertificadoVigente(x)	El certificado digital para firmar x está vigente
ResolucionVigente(x)	La resolución de numeración DIAN está vigente
NumeroEnRango(x)	El número de x está en el rango autorizado
TerceroOk(x)	Los datos del tercero en x son correctos
PropietarioOk(x)	Los datos del propietario del vehículo en x son co
ClienteOk(x)	Los datos del cliente en x son correctos
DestinatarioOk(x)	Los datos del destinatario en x son correctos
RemitenteOk(x)	Los datos del remitente en x son correctos
VehiculoOk(x)	Los datos del vehículo en x son correctos
FacturableOk(x)	Los datos del facturado en x son correctos
CoordenadasCoherentes(x)	Las coordenadas GPS del destino de x son coherente
DestinoExisteRNDC(x)	El municipio de destino de x existe en el catálogo
FacturaAceptadaCliente(x)	La factura asociada a x ya fue aceptada por el cli
DocumentoTipo12(x)	x es un documento de tipo 12 (transporte)
FacturableConsistente(x)	Los datos del facturable en el XML coinciden con l

Predicados de diagnóstico (conclusión)

Predicado	Significado
DianCaida(x)	La DIAN no está disponible
RndcCaida(x)	El RNDC no está disponible
ErrorAPI(x)	Problema en la integración con la API

ErrorCertificado(x)	El certificado digital no está vigente
ErrorResolucionNumeracion(x)	La resolución de numeración no está vigente
ErrorNumeroFueraRango(x)	El número de factura está fuera del rango
DocumentoYaProcesado(x)	El documento ya fue registrado previamente
ProblemaInternet(x)	Problema de conectividad a internet
TerceroMalDiligenciado(x)	Datos del tercero incorrectos
PropietarioMalDiligenciado(x)	Datos del propietario incorrectos
ClienteMalDiligenciado(x)	Datos del cliente incorrectos
DestinatarioMalDiligenciado(x)	Datos del destinatario incorrectos
VehiculoMalDiligenciado(x)	Datos del vehículo incorrectos
FacturableMalDiligenciado(x)	Datos del facturable incorrectos
ErrorCoordenadasRemesa(x)	Coordenadas incoherentes con la ciudad
DestinoNoExisteRNDC(x)	El destino no existe en el catálogo RNDC
ErrorFacturaYaAceptada(x)	Factura ya aceptada - no se puede anular
ErrorDocumentoNoTipo12(x)	El documento enviado al RNDC no es tipo 12
ErrorFacturableInconsistente(x)	Facturable en XML inconsistente con la remesa
EnvioExitosoRNDC(x)	El RNDC aceptó el manifiesto
CufeGenerado(x)	La DIAN generó el CUFE exitosamente
TipoXML12(x)	La DIAN respondió con XML tipo 12
TipoXML10(x)	La DIAN respondió con XML tipo 10

3.2 Reglas en lógica de predicados (selección representativa)

R1a - DIAN caída:

```
para todo x (Factura(x) Y EnvioDian(x) Y FacturableOk(x) Y NOTErrorConexion(x) Y TieneInternet(x)
  Y CertificadoVigente(x) Y ResolucionVigente(x) Y NumeroEnRango(x)
  Y DocumentoRechazado(x) Y NOTTimeoutEnvio(x) Y NOTFallaSOAP(x) -> DianCaída(x))
```

R7a - Problema de internet:

```
para todo x (EnvioDian(x) Y NOTTieneInternet(x) -> ProblemaInternet(x))
```

R15 - Manifiesto exitoso:

```
para todo x (Manifiesto(x) Y EnvioRNDC(x) Y TerceroOk(x) Y FacturableOk(x) Y PropietarioOk(x)
  Y ClienteOk(x) Y DestinatarioOk(x) Y RemitenteOk(x) Y VehiculoOk(x)
  Y TieneInternet(x) Y NOTFallaSOAP(x) -> EnvioExitosoRNDC(x))
```

3.3 Reglas en formato SI-ENTONCES (selección)

```

R1a [Certeza: 0.90]
SI Factura(x) Y EnvioDian(x) Y FacturableOk(x) Y NO ErrorConexion(x)
  Y TieneInternet(x) Y CertificadoVigente(x) Y ResolucionVigente(x)
  Y NumeroEnRango(x) Y DocumentoRechazado(x) Y NO TimeoutEnvio(x) Y NO FallaSOAP(x)
ENTONCES DianCaida(x)

R7a [Certeza: 1.00]
SI EnvioDian(x) Y NO TieneInternet(x)
ENTONCES ProblemaInternet(x)

R4a [Certeza: 1.00]
SI Factura(x) Y EnvioDian(x) Y NO CertificadoVigente(x)
ENTONCES ErrorCertificado(x)

R16a [Certeza: 0.98]
SI Factura(x) Y EnvioDian(x) Y CertificadoVigente(x) Y ResolucionVigente(x)
  Y NumeroEnRango(x) Y FacturableOk(x)
ENTONCES CufeGenerado(x)

```

3.4 Organización de reglas por grupo

Grupo	Código	N° Reglas	Dominio
A - Conectividad	R1a, R1b, R2a, R2b, R3a, R3b, R3c	8	Disponibilidad del servicio e internet
B - Certificado	R4a, R4b	2	Firma digital DIAN
C - Numeración	R5a, R5b	2	Resolución y rango de numeración
D - Duplicado	R6	1	Documentos ya procesados
E - Manifiesto RNDC	R10a-R10e	5	Entidades en manifiestos
F - Remesa RNDC	R11a-R11d	4	Entidades en remesas
G - Factura DIAN	R12	1	Facturable en facturas
H - NC DIAN	R13a, R13b	2	Nota crédito y factura aceptada
I - Factura/NC RNDC	R14a-R14d	4	Tipo 12 y consistencia XML
J - Éxito	R15, R16a, R16b, R17a-R17d	6	Envíos correctos
Total		35	

4. Motor de inferencia

4.1 Algoritmo de encadenamiento hacia adelante


```

FUNCIÓN encadenamiento_adelante(reglas, hechos):
1. Eliminar hechos duplicados preservando orden
2. validar_hechos() - detectar contradicciones y tipos mutuamente excluyentes
3. aplicar_cwa() - agregar (pred, False) para predicados no mencionados
4. aplicadas <- []
5. certezas <- {}
6. cambio <- True
7. iteracion <- 0
8. MIENTRAS cambio Y iteracion < MAX_ITER:
    cambio <- False
    iteracion <- iteracion + 1
    PARA CADA regla EN reglas:
        SI regla.nombre EN aplicadas: CONTINUAR (anti-ciclo)
        SI todas las condiciones están en hechos:
            aplicadas.agregar(regla.nombre)
            SI conclusion no está en hechos:
                hechos.agregar(conclusion)
                certezas[conclusion] <- regla.certeza
                cambio <- True
            SINO:
                certezas[conclusion] <- combinar(certezas[conclusion], regla.certeza)
9. RETORNAR (hechos, aplicadas, certezas)

```

Prevención de ciclos infinitos: cada regla se marca en `aplicadas` al dispararse y no se vuelve a evaluar. El bucle termina en a lo sumo `len(reglas)` iteraciones. Se agrega una salvaguarda `max_iter = len(reglas) + 1`.

4.2 Supuesto de Mundo Cerrado (CWA)

Todo predicado de síntoma no mencionado explícitamente se asume `False`:

```

def aplicar_cwa(hechos):
    presentes = {h[0] for h in hechos}
    return list(hechos) + [(p, False) for p in PREDICADOS_SINTOMA if p not in presentes]

```

Esto permite que el motor infiera correctamente aún con información parcial.

4.3 Combinación de certezas

Cuando múltiples reglas derivan la misma conclusión se combina usando:

$$P(A \cup B) = 1 - (1 - P(A)) \cdot (1 - P(B))$$

Ejemplo: R1a y R1b ambas derivan `DianCaida` con certeza 0.90:

$$P = 1 - (1 - 0.90) \cdot (1 - 0.90) = 1 - 0.01 = 0.99$$

4.4 Función `porque()`

Explica por qué se llegó a un diagnóstico específico devolviendo las reglas aplicadas que lo derivaron y sus condiciones:

```
def porque(conclusion, reglas, reglas_aplicadas, hechos_finales):
    return [
        {
            "regla": regla["nombre"],
            "certeza": regla["certeza"],
            "descripcion": DESCRIPCIONES_REGLAS[regla["nombre"]],
            "condiciones": [{"predicado": p, "valor_esperado": v}
                           for p, v in regla["condiciones"]],
        }
        for regla in reglas
        if regla["nombre"] in reglas_aplicadas
        and regla["conclusion"][0] == conclusion
    ]
```

4.5 Logging del proceso

El motor registra cada paso usando el módulo estándar `logging`:

- Inicio de inferencia con número de hechos.
- Cada regla disparada con su conclusión y certeza.
- Combinaciones de certeza.
- Advertencia si se alcanza el límite de iteraciones.

5. Implementación técnica

5.1 Tecnologías utilizadas

Tecnología	Versión	Uso
Python	3.8+	Lenguaje principal
Streamlit	>=1.26	Interfaz web interactiva
logging	stdlib	Trazabilidad del motor

5.2 Decisiones de diseño

Separación de responsabilidades: el código está dividido en tres módulos independientes. `base_conocimiento.py` contiene solo datos (sin lógica), `motor_inferencia.py` contiene solo lógica (sin UI), y `app.py` contiene solo la presentación. Esto permite reutilizar el motor en otros contextos (CLI, API REST, etc.) sin

modificarlo.

CWA vs. preguntar al usuario: se eligió CWA sobre pedir confirmación de cada predicado porque los usuarios no técnicos no conocen todos los predicados del sistema. El CWA simplifica la interfaz sin perder precisión.

Streamlit sobre Flask: Streamlit permite construir la interfaz completa en un solo archivo Python sin necesidad de HTML/CSS separado, reduciendo la complejidad de despliegue y mantenimiento.

Certezas independientes por regla: cada regla tiene su propio factor de certeza asignado por análisis de dominio. Las certezas más bajas (0.90) corresponden a diagnósticos donde el síntoma podría tener otras causas no cubiertas por el sistema.

5.3 Desafíos encontrados y soluciones

Desafío	Solución
CSS de Streamlit sobrescribía texto del hero head	Se colocaron las reglas del hero al final del blog
<div> de Markdown no envuelve widgets Streamlit	Se eliminaron los wrappers y se usaron encabezados
El CWA disparaba reglas irrelevantes	Las reglas incluyen el tipo de documento y destino
Texto invisible en checkboxes	Se añadieron reglas CSS explícitas para todos los

6. Validación

6.1 Resumen de casos de prueba

Ver tabla completa en [tests/casos_prueba.md](#).

Aspecto	Resultado
Total de casos documentados	12
Casos con diagnóstico de error	9
Casos de éxito	2 (casos 8, 9, 10)
Caso sin conclusión	1 (caso 12)
Todos los casos pasan	[SI]

6.2 Análisis de resultados

El motor demuestra correctitud en todos los casos:

- **Casos simples** (un síntoma -> un diagnóstico): casos 2, 3, 5, 6, 7, 11.
- **Casos compuestos** (múltiples síntomas -> múltiples diagnósticos): caso 4 (tres errores DIAN).

- **Encadenamiento en cadena:** casos 9 y 10 (R16a -> R17a/R17b).
- **Caso límite sin conclusión:** caso 12 confirma que el sistema no produce falsos positivos.

6.3 Limitaciones identificadas

- El sistema no cubre errores internos del software (bugs propios de la aplicación).
- No modela fallos parciales de red (latencia alta sin timeout completo).
- El CWA puede generar diagnósticos de error cuando una entidad no fue mencionada por el usuario pero debería ser relevante; esto se mitiga con la interfaz que solo muestra las entidades pertinentes al tipo de documento seleccionado.

7. Conclusiones

7.1 Logros alcanzados

- Se construyó un motor de inferencia funcional con 35 reglas y 28 predicados.
- La interfaz web guía al usuario en 3 pasos simples con preguntas en lenguaje natural.
- La función `porque()` proporciona trazabilidad completa por diagnóstico individual.
- El sistema maneja correctamente casos de éxito, error y sin conclusión.
- El código está modularizado y documentado según estándares profesionales.

7.2 Aprendizajes obtenidos

- La formalización en lógica de predicados antes de codificar las reglas evitó ambigüedades y redujo errores de implementación.
- El supuesto de mundo cerrado es una herramienta poderosa pero debe aplicarse con cuidado: es necesario asegurar que las reglas incluyan suficientes condiciones de contexto para no generar falsos positivos.
- La separación backend/frontend desde el inicio facilitó las iteraciones de diseño de la interfaz sin afectar la lógica del motor.

7.3 Posibles mejoras futuras

- Agregar encadenamiento hacia atrás para consultas del tipo "¿qué síntomas causarían X?".
- Integrar con la API de estado de servicios de la DIAN para validar en tiempo real si el servicio está caído.
- Ampliar el dominio para cubrir errores en el módulo de nómina electrónica.
- Añadir historial de diagnósticos por empresa/usuario con persistencia en base de datos.